Below is a **consolidated, production-oriented Selenium + Java + TestNG framework** with representative sample code for the major layers we discussed.

I'll use **Selenium Java 4.46.0**, which is the current stable Selenium release as of July 2026. Selenium's official documentation also recommends managing the Java binding through Maven. (Selenium) For TestNG, I'll use **7.11.0**; TestNG's current documentation lists 7.9.0 in its Maven example, while its official update site lists 7.11.0, so pinning the version explicitly in your project is preferable. (TestNG)

1. Final framework architecture

```
selenium-testng-framework/
|
├── pom.xml
|
├── src/
|   |
|   ├── main/java/com/automation/framework/
|   |   |
|   |   ├── config/
|   |   |   └── ConfigManager.java
|   |   |
|   |   ├── driver/
|   |   |   ├── DriverFactory.java
|   |   |   └── DriverManager.java
|   |   |
|   |   ├── pages/
|   |   |   ├── BasePage.java
|   |   |   ├── LoginPage.java
|   |   |   └── HomePage.java
|   |   |
|   |   ├── components/
|   |   |   └── HeaderComponent.java
|   |   |
|   |   ├── data/
|   |   |   ├── ExcelReader.java
```

```
|   |   |   ├── JsonReader.java
|   |   |   └── DataProviderFactory.java
|   |   |
|   |   ├── utils/
|   |   |   ├── WaitUtils.java
|   |   |   ├── ScreenshotUtils.java
|   |   |   └── JsonUtils.java
|   |   |
|   |   └── listeners/
|   |       └── TestListener.java
|   |
|   └── test/java/com/automation/tests/
|       |
|       ├── base/
|       |   └── BaseTest.java
|       |
|       ├── login/
|       |   └── LoginTest.java
|       |
|       └── checkout/
|           └── CheckoutTest.java
|
├── src/test/resources/
|   |
|   ├── config/
|   |   ├── qa.properties
|   |   └── staging.properties
|   |
|   ├── testdata/
|   |   └── login.xlsx
|   |
```

```
|   └── testng/
|       ├── smoke.xml
|       └── regression.xml
|
└── reports/
```

The dependency direction should remain:

Test

↓

Page Object / Component

↓

BasePage

↓

DriverManager

↓

Selenium WebDriver

while data flows independently:

Excel / JSON / DB

↓

Data Reader

↓

DataProvider

↓

Test

---

2. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="
         http://maven.apache.org/POM/4.0.0
```

```
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.company</groupId>
<artifactId>selenium-testng-framework</artifactId>
<version>1.0.0</version>

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>

  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  <selenium.version>4.46.0</selenium.version>
  <testng.version>7.11.0</testng.version>
  <poi.version>5.4.1</poi.version>
  <jackson.version>2.19.2</jackson.version>
  <slf4j.version>2.0.17</slf4j.version>
  <logback.version>1.5.18</logback.version>
</properties>

<dependencies>

  <!-- Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>${selenium.version}</version>
  </dependency>
```

```
<!-- TestNG -->
<dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
    <scope>test</scope>
</dependency>

<!-- Apache POI - Excel -->
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>${poi.version}</version>
</dependency>

<!-- Jackson - JSON -->
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>${jackson.version}</version>
</dependency>

<!-- Logging -->
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>${slf4j.version}</version>
</dependency>

<dependency>
    <groupId>ch.qos.logback</groupId>
```

```
        <artifactId>logback-classic</artifactId>
        <version>${logback.version}</version>
    </dependency>

</dependencies>

<build>

    <plugins>

        <!-- Compiler -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.14.0</version>

            <configuration>
                <source>17</source>
                <target>17</target>
            </configuration>
        </plugin>

        <!-- Surefire -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.5.3</version>

            <configuration>
                <suiteXmlFiles>
                    <suiteXmlFile>
```

```
            src/test/resources/testng/regression.xml
          </suiteXmlFile>
        </suiteXmlFiles>

        <systemPropertyVariables>
          <env>${env}</env>
          <browser>${browser}</browser>
        </systemPropertyVariables>

      </configuration>
    </plugin>

  </plugins>

</build>

</project>
```

Selenium officially supports Maven-based Java setup, and Selenium 4.46.0 is the current stable release at the time of writing. (Selenium)

---

3. Configuration

qa.properties

```
baseUrl=https://qa.example.com

browser=chrome
headless=false

explicitWait=15
pageLoadTimeout=30
scriptTimeout=30
```

```
remoteExecution=false
gridUrl=http://localhost:4444
```

---

4. ConfigManager.java

```
package com.automation.framework.config;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

public final class ConfigManager {

   private static final Properties properties = new Properties();

   static {

      String environment =
            System.getProperty("env", "qa");

      String fileName =
            "config/" + environment + ".properties";

      try (InputStream input =
               ConfigManager.class
                     .getClassLoader()
                     .getResourceAsStream(fileName)) {

         if (input == null) {
            throw new RuntimeException(
                  "Configuration file not found: " + fileName);
         }
```

```
            properties.load(input);

        } catch (IOException e) {
            throw new RuntimeException(
                    "Unable to load configuration", e);
        }
    }

    private ConfigManager() {
    }

    public static String get(String key) {

        String systemValue =
                System.getProperty(key);

        if (systemValue != null) {
            return systemValue;
        }

        return properties.getProperty(key);
    }

    public static int getInt(String key) {
        return Integer.parseInt(get(key));
    }

    public static boolean getBoolean(String key) {
        return Boolean.parseBoolean(get(key));
    }
```

}

This gives you command-line overrides:

mvn clean test -Denv=qa -Dbrowser=firefox

---

5. Thread-safe Driver Manager

This is one of the most important parts of the framework.

DriverManager.java

```
package com.automation.framework.driver;

import org.openqa.selenium.WebDriver;

public final class DriverManager {

   private static final ThreadLocal<WebDriver> DRIVER =
         new ThreadLocal<>();

   private DriverManager() {
   }

   public static void setDriver(WebDriver driver) {
      DRIVER.set(driver);
   }

   public static WebDriver getDriver() {

      WebDriver driver = DRIVER.get();

      if (driver == null) {
         throw new IllegalStateException(
               "WebDriver has not been initialized for this thread.");
      }
```

```
        return driver;
    }

    public static void quitDriver() {

        WebDriver driver = DRIVER.get();

        if (driver != null) {
            driver.quit();
            DRIVER.remove();
        }
    }
}
```

This allows:

Thread 1 → Driver A

Thread 2 → Driver B

Thread 3 → Driver C

Thread 4 → Driver D

which is essential for parallel TestNG execution.

---

6. Driver Factory

DriverFactory.java

```
package com.automation.framework.driver;

import com.automation.framework.config.ConfigManager;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.firefox.FirefoxDriver;
```

```
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.edge.EdgeOptions;
import org.openqa.selenium.remote.DesiredCapabilities;
import org.openqa.selenium.remote.RemoteWebDriver;

import java.net.MalformedURLException;
import java.net.URI;

public final class DriverFactory {

   private DriverFactory() {
   }

   public static WebDriver createDriver() {

      String browser =
            ConfigManager.get("browser").toLowerCase();

      boolean headless =
            ConfigManager.getBoolean("headless");

      boolean remote =
            ConfigManager.getBoolean("remoteExecution");

      if (remote) {
         return createRemoteDriver(browser, headless);
      }

      return createLocalDriver(browser, headless);
   }
```

```
private static WebDriver createLocalDriver(
        String browser,
        boolean headless) {

    return switch (browser) {

        case "chrome" ->
                createChrome(headless);

        case "firefox" ->
                createFirefox(headless);

        case "edge" ->
                createEdge(headless);

        default ->
                throw new IllegalArgumentException(
                        "Unsupported browser: " + browser);
    };
}

private static WebDriver createChrome(
        boolean headless) {

    ChromeOptions options =
            new ChromeOptions();

    if (headless) {
        options.addArguments("--headless=new");
    }
```

```
    options.addArguments(
            "--start-maximized",
            "--disable-notifications");

    return new ChromeDriver(options);
}

private static WebDriver createFirefox(
        boolean headless) {

    FirefoxOptions options =
            new FirefoxOptions();

    if (headless) {
        options.addArguments("-headless");
    }

    return new FirefoxDriver(options);
}

private static WebDriver createEdge(
        boolean headless) {

    EdgeOptions options =
            new EdgeOptions();

    if (headless) {
        options.addArguments("--headless=new");
    }
```

```
    return new EdgeDriver(options);
}

private static WebDriver createRemoteDriver(
        String browser,
        boolean headless) {

    try {

        String gridUrl =
                ConfigManager.get("gridUrl");

        return switch (browser) {

            case "chrome" -> {

                ChromeOptions options =
                        new ChromeOptions();

                if (headless) {
                    options.addArguments("--headless=new");
                }

                yield new RemoteWebDriver(
                        URI.create(gridUrl).toURL(),
                        options);
            }

            case "firefox" -> {

                FirefoxOptions options =
```

```
                new FirefoxOptions();

            if (headless) {
                options.addArguments("-headless");
            }

            yield new RemoteWebDriver(
                URI.create(gridUrl).toURL(),
                options);
        }

        case "edge" -> {

            EdgeOptions options =
                new EdgeOptions();

            if (headless) {
                options.addArguments("--headless=new");
            }

            yield new RemoteWebDriver(
                URI.create(gridUrl).toURL(),
                options);
        }

        default ->
            throw new IllegalArgumentException(
                "Unsupported browser: " + browser);
    };

} catch (MalformedURLException e) {
```

```
            throw new RuntimeException(
                "Invalid Selenium Grid URL", e);
        }
    }
}
```

Selenium Grid is specifically intended for distributing tests across machines and browser/OS combinations, making it the natural next step when local parallel execution is no longer enough. (Selenium)

---

7. Base Test

BaseTest.java

```
package com.automation.tests.base;

import com.automation.framework.config.ConfigManager;
import com.automation.framework.driver.DriverFactory;
import com.automation.framework.driver.DriverManager;

import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public abstract class BaseTest {

    @BeforeMethod(alwaysRun = true)
    public void setUp() {

        WebDriver driver =
                DriverFactory.createDriver();

        DriverManager.setDriver(driver);

        driver.manage()
```

```
            .timeouts()
            .pageLoadTimeout(
                  java.time.Duration.ofSeconds(
                        ConfigManager.getInt(
                              "pageLoadTimeout")));

      driver.manage()
            .timeouts()
            .scriptTimeout(
                  java.time.Duration.ofSeconds(
                        ConfigManager.getInt(
                              "scriptTimeout")));

      driver.get(
            ConfigManager.get("baseUrl"));
   }

   @AfterMethod(alwaysRun = true)
   public void tearDown() {

      DriverManager.quitDriver();
   }
}
```

---

8. Wait utility

WaitUtils.java

```
package com.automation.framework.utils;

import com.automation.framework.config.ConfigManager;
import com.automation.framework.driver.DriverManager;
```

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

public final class WaitUtils {

    private WaitUtils() {
    }

    private static WebDriverWait getWait() {

        return new WebDriverWait(
                DriverManager.getDriver(),
                Duration.ofSeconds(
                        ConfigManager.getInt(
                                "explicitWait")));
    }

    public static WebElement visible(By locator) {

        return getWait().until(
                ExpectedConditions.visibilityOfElementLocated(
                        locator));
    }

    public static WebElement clickable(By locator) {

        return getWait().until(
```

```
            ExpectedConditions.elementToBeClickable(
                    locator));
    }

    public static boolean urlContains(
            String value) {

        return getWait().until(
                ExpectedConditions.urlContains(value));
    }
}
```

The framework should use explicit waits rather than scattering Thread.sleep() throughout tests.

---

9. Base Page

BasePage.java

```
package com.automation.framework.pages;

import com.automation.framework.driver.DriverManager;
import com.automation.framework.utils.WaitUtils;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

public abstract class BasePage {

    protected final WebDriver driver;

    protected BasePage() {
        this.driver =
                DriverManager.getDriver();
```

```
}

protected void click(By locator) {

    WaitUtils.clickable(locator)
            .click();
}

protected void type(
        By locator,
        String value) {

    WebElement element =
            WaitUtils.visible(locator);

    element.clear();
    element.sendKeys(value);
}

protected String getText(By locator) {

    return WaitUtils.visible(locator)
            .getText();
}

protected boolean isDisplayed(
        By locator) {

    try {
        return WaitUtils.visible(locator)
                .isDisplayed();
```

```
        } catch (Exception e) {
            return false;
        }
    }
}
```

Now Page Objects don't need to repeatedly implement basic Selenium operations.

---

10. Login Page

LoginPage.java

```
package com.automation.framework.pages;

import org.openqa.selenium.By;

public class LoginPage extends BasePage {

    private final By username =
            By.id("username");

    private final By password =
            By.id("password");

    private final By loginButton =
            By.id("login");

    private final By errorMessage =
            By.cssSelector(".error-message");

    public HomePage login(
            String usernameValue,
            String passwordValue) {
```

```
        type(username, usernameValue);
        type(password, passwordValue);

        click(loginButton);

        return new HomePage();
    }

    public String getErrorMessage() {

        return getText(errorMessage);
    }
}
```

Notice that the test doesn't know anything about the locators.

---

11. Home Page

HomePage.java

```
package com.automation.framework.pages;

import org.openqa.selenium.By;

public class HomePage extends BasePage {

    private final By homePageIdentifier =
            By.cssSelector(".dashboard");

    public boolean isDisplayed() {

        return isDisplayed(
                homePageIdentifier);
```

```
    }
}
```

---

12. Data model

Instead of passing five or six strings around, create a model.

LoginData.java

```
package com.automation.framework.data;

public record LoginData(
        String username,
        String password,
        String expectedResult) {
}
```

This makes your tests easier to understand.

---

13. Excel Reader

ExcelReader.java

```
package com.automation.framework.data;

import org.apache.poi.ss.usermodel.*;

import java.io.FileInputStream;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public final class ExcelReader {

    private ExcelReader() {
    }
```

```
public static List<LoginData> readLoginData(
    String filePath,
    String sheetName) {

  List<LoginData> data =
      new ArrayList<>();

  try (FileInputStream input =
          new FileInputStream(filePath);

   Workbook workbook =
          WorkbookFactory.create(input)) {

    Sheet sheet =
        workbook.getSheet(sheetName);

    for (int i = 1;
      i <= sheet.getLastRowNum();
      i++) {

      Row row = sheet.getRow(i);

      if (row == null) {
        continue;
      }

      String username =
          getCellValue(row.getCell(0));

      String password =
          getCellValue(row.getCell(1));
```

```
            String expected =
                    getCellValue(row.getCell(2));

            data.add(
                    new LoginData(
                            username,
                            password,
                            expected));
          }

      } catch (IOException e) {

          throw new RuntimeException(
                  "Unable to read Excel file", e);
      }

      return data;
   }

   private static String getCellValue(
          Cell cell) {

      if (cell == null) {
          return "";
      }

      return cell.toString().trim();
   }
}
```

14. TestNG DataProvider

DataProviderFactory.java

```
package com.automation.framework.data;

import org.testng.annotations.DataProvider;

import java.util.List;

public class DataProviderFactory {

   @DataProvider(name = "loginData")
   public static Object[][] loginData() {

      List<LoginData> data =
            ExcelReader.readLoginData(
                  "src/test/resources/testdata/login.xlsx",
                  "Login");

      return data.stream()
            .map(item -> new Object[]{item})
            .toArray(Object[][]::new);
   }
}
```

---

15. Login test

LoginTest.java

```
package com.automation.tests.login;

import com.automation.framework.data.DataProviderFactory;
import com.automation.framework.data.LoginData;
import com.automation.framework.pages.HomePage;
```

```
import com.automation.framework.pages.LoginPage;
import com.automation.tests.base.BaseTest;

import org.testng.Assert;
import org.testng.annotations.Test;

public class LoginTest extends BaseTest {

   @Test(
      dataProvider = "loginData",
      dataProviderClass = DataProviderFactory.class,
      groups = {"smoke", "regression"}
   )
   public void loginTest(LoginData data) {

      LoginPage loginPage =
            new LoginPage();

      HomePage homePage =
            loginPage.login(
                  data.username(),
                  data.password());

      if (data.expectedResult()
            .equalsIgnoreCase("SUCCESS")) {

         Assert.assertTrue(
               homePage.isDisplayed(),
               "Home page should be displayed");

      } else {
```

```
            Assert.assertFalse(
                    homePage.isDisplayed(),
                    "Home page should not be displayed");
        }
    }
}
```

The resulting architecture is:

```
login.xlsx
   ↓
ExcelReader
   ↓
LoginData
   ↓
@DataProvider
   ↓
LoginTest
   ↓
LoginPage
   ↓
HomePage
```

That's the core of the **data-driven framework**.

---

16. TestNG Listener

TestListener.java

```
package com.automation.framework.listeners;

import com.automation.framework.utils.ScreenshotUtils;

import org.testng.ITestListener;
import org.testng.ITestResult;
```

```
public class TestListener
      implements ITestListener {

   @Override
   public void onTestStart(
         ITestResult result) {

      System.out.println(
            "START: " +
            result.getName());
   }

   @Override
   public void onTestSuccess(
         ITestResult result) {

      System.out.println(
            "PASS: " +
            result.getName());
   }

   @Override
   public void onTestFailure(
         ITestResult result) {

      System.out.println(
            "FAIL: " +
            result.getName());

      ScreenshotUtils.capture(
```

```
            result.getName());
    }

    @Override
    public void onTestSkipped(
            ITestResult result) {

        System.out.println(
            "SKIP: " +
            result.getName());
    }
}
```

---

17. Screenshot utility

ScreenshotUtils.java

```
package com.automation.framework.utils;

import com.automation.framework.driver.DriverManager;

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;

import java.io.File;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

public final class ScreenshotUtils {

    private ScreenshotUtils() {
    }
```

```
public static String capture(
        String testName) {

    try {

        File source =
                ((TakesScreenshot)
                        DriverManager.getDriver())
                        .getScreenshotAs(
                                OutputType.FILE);

        Path destination =
                Path.of(
                        "reports",
                        "screenshots",
                        testName + "_" +
                        System.currentTimeMillis() +
                        ".png");

        Files.createDirectories(
                destination.getParent());

        Files.copy(
                source.toPath(),
                destination,
                StandardCopyOption.REPLACE_EXISTING);

        return destination.toString();

    } catch (Exception e) {
```

```
            System.err.println(
                    "Screenshot failed: " +
                    e.getMessage());

            return null;
        }
    }
}
```

---

18. Register listener

You can register the listener in testng.xml:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM
        "https://testng.org/testng-1.0.dtd">

<suite name="Regression Suite">

    <listeners>
        <listener
            class-name=
            "com.automation.framework.listeners.TestListener"/>
    </listeners>

    <test name="Login Tests">

        <classes>

            <class
                name="com.automation.tests.login.LoginTest"/>
```

```
    </classes>

  </test>

</suite>
```

TestNG organizes execution through suites, tests, classes, groups, and annotations; the suite XML is the standard way to define these execution units. (TestNG)

---

19. Parallel TestNG suite

For scalable execution:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM
    "https://testng.org/testng-1.0.dtd">

<suite
  name="Parallel Regression"
  parallel="classes"
  thread-count="4">

  <listeners>
    <listener
      class-name=
      "com.automation.framework.listeners.TestListener"/>
  </listeners>

  <test name="Regression">

    <classes>
```

```
        <class
           name="com.automation.tests.login.LoginTest"/>

        <class
           name="com.automation.tests.checkout.CheckoutTest"/>

      </classes>

   </test>

</suite>
```

Combined with ThreadLocal<WebDriver>:

```
              TestNG
                |
        ┌───────┼───────┐
        |       |       |
     Thread 1  Thread 2  Thread 3
        |       |       |
     Driver A  Driver B  Driver C
        |       |       |
     Chrome    Chrome    Chrome
```

---

20. Smoke suite

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM
      "https://testng.org/testng-1.0.dtd">

<suite name="Smoke Suite">

   <test name="Smoke Tests">
```

```
    <groups>
      <run>
        <include name="smoke"/>
      </run>
    </groups>

    <classes>

      <class
        name="com.automation.tests.login.LoginTest"/>

    </classes>

  </test>

</suite>
```

Run:

```
mvn clean test
```

Or:

```
mvn clean test -Denv=qa -Dbrowser=chrome
```

---

21. Remote Selenium Grid

Your configuration can switch from local to Grid:

```
remoteExecution=true
gridUrl=http://localhost:4444
```

Then the same test executes:

Test

↓

BaseTest

↓

DriverFactory

↓

RemoteWebDriver

↓

Selenium Grid

↓

Browser Node

No changes are required to:

LoginTest

LoginPage

HomePage

DataProvider

That is exactly what we want from a scalable framework.

---

22. Optional component architecture

For a large application, use components.

HeaderComponent.java

```
package com.automation.framework.components;


import com.automation.framework.pages.BasePage;


import org.openqa.selenium.By;


public class HeaderComponent
      extends BasePage {


   private final By logout =
         By.id("logout");


   private final By profile =
         By.id("profile");
```

```
    public void logout() {
        click(logout);
    }

    public void openProfile() {
        click(profile);
    }
}
```

Then:

```
public class HomePage extends BasePage {

    private final HeaderComponent header =
            new HeaderComponent();

    public void logout() {
        header.logout();
    }
}
```

This prevents common UI elements from being duplicated across dozens of Page Objects.

---

23. Recommended test-data structure

For Excel:

login.xlsx

Sheet: Login

```
-----------------------------------------------------
username        password       expectedResult
-----------------------------------------------------
valid@test.com    Password123    SUCCESS
```

locked@test.com Password123 FAILURE

invalid@test.com WrongPassword FAILURE

For larger systems, I'd eventually move away from putting all data into Excel.

A better long-term strategy is:

Small/static data

↓

JSON

Business-managed datasets

↓

Excel

Large/dynamic datasets

↓

Database/API

Secrets

↓

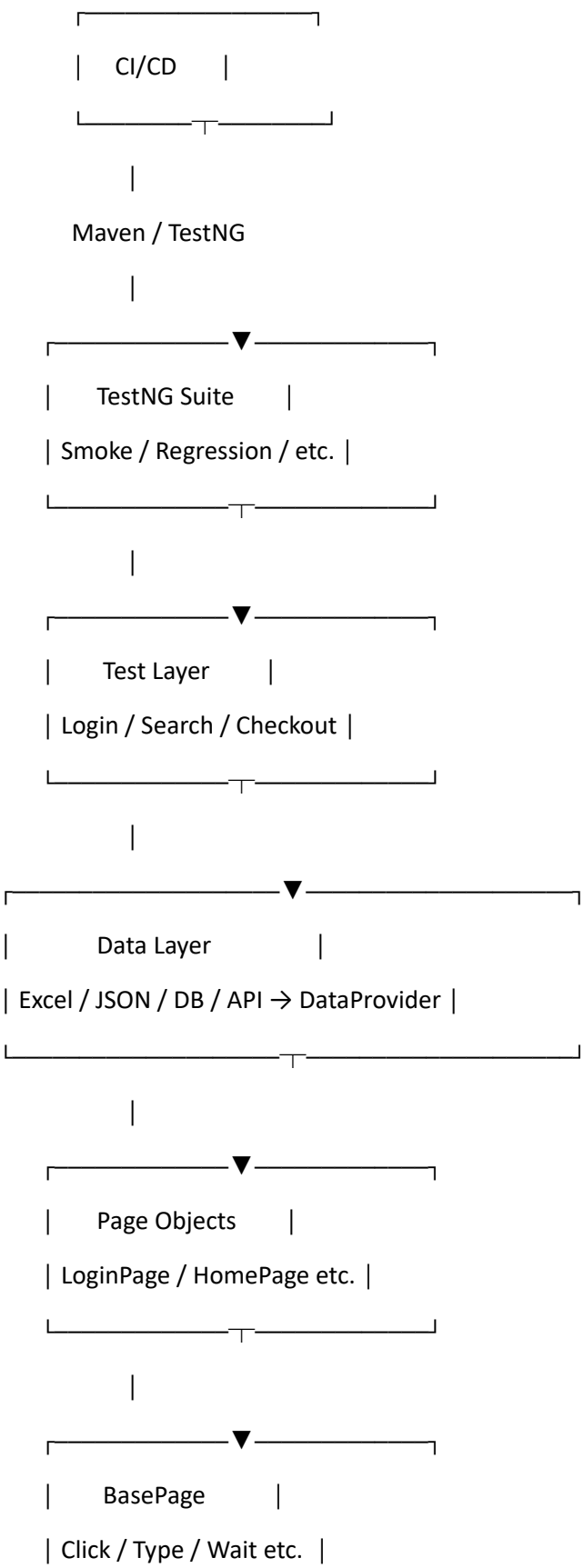
CI/CD secret store

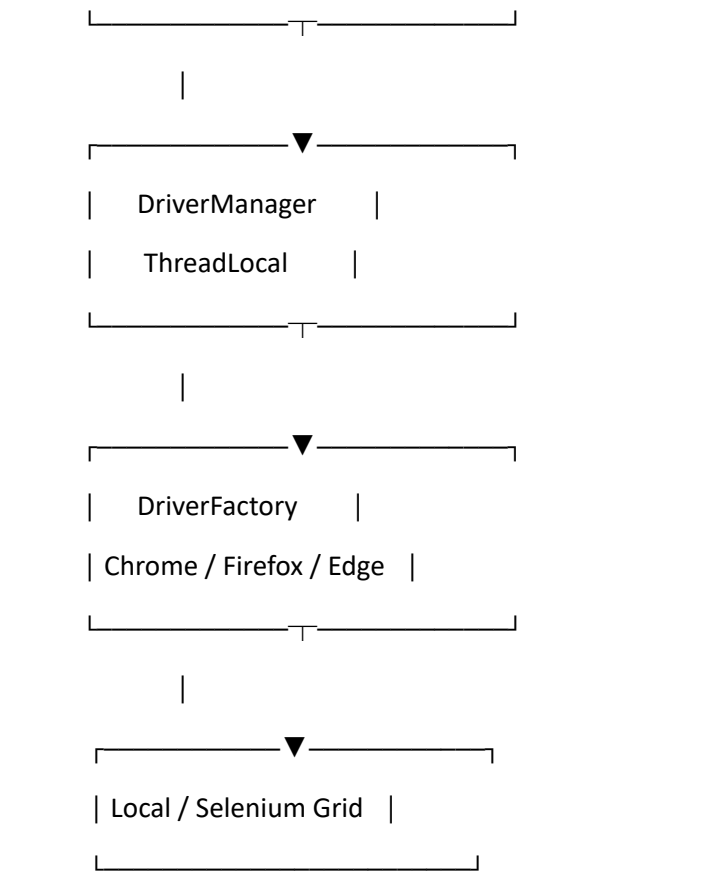
Never put real passwords, API keys, tokens, or other secrets in Excel, JSON, Git, or properties files.

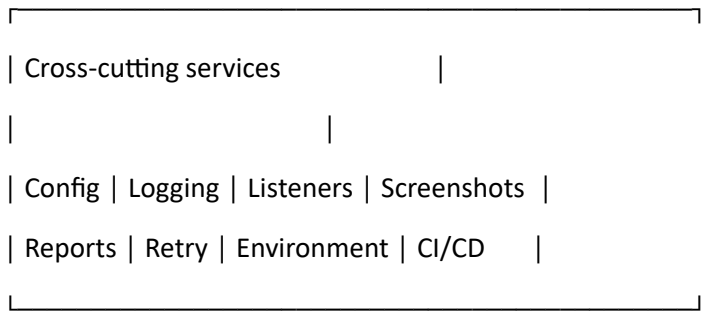
---

24. Final production flow

The complete framework now looks like this:







Build order for this implementation

I would implement it in exactly this sequence:

01 Maven + project structure

02 pom.xml

03 ConfigManager

04 environment properties

05 DriverManager + ThreadLocal

06 DriverFactory

07 BaseTest

08 WaitUtils

09 BasePage

10 Page Objects

11 Components

12 Data models

13 Excel/JSON readers

14 TestNG DataProviders

15 Test classes

16 TestNG groups/suites

17 TestNG listeners

18 Screenshots

19 Logging

20 Reporting

21 Retry handling

22 Parallel execution

23 Selenium Grid

24 CI/CD

25 Docker/Grid scaling

The most important design rules

Keep these five boundaries clean:

Test → Business validation

Page → UI behavior

DataProvider → Test data

Driver → Browser lifecycle

Listener → Execution/reporting concerns

That separation is what makes the framework **maintainable, data-driven, thread-safe, parallelizable, and ready for Selenium Grid/CI/CD** rather than becoming a collection of Selenium scripts.

For a real enterprise implementation, the next logical step would be to turn this into a **complete runnable Git-style project**, including every Java class, pom.xml, testng.xml, Excel reader, JSON reader, logging configuration, retry analyzer, Extent/Allure reporting, Grid configuration, and sample Login/Checkout modules.